

Month Report & Tracker

Microservices Application

Employee Name	Abhishek Gande
Project	Microservices Application
Team Lead	Yenikepalli Sai kumar

Purpose of This Document

This document provides a clear record of the assignments and tasks completed during the project. Each task follows a simple work lifecycle: Objective → Work Completed → Validation → Outcome.

Technical Skills Learnt

- CRUD Operations and RESTful API design and implementation
- Java and the Spring Boot development workflow
- Microservices Architecture and database-per-service design
- MySQL — schema design, indexing, and query optimization
- API documentation and versioning using Swagger/OpenAPI
- Authentication and authorization using Spring Security, JWT access/refresh tokens, and role-based access control
- Inter-service communication patterns, including timeouts, retries, Circuit Breaker, and fallback handling
- Event-driven architecture and asynchronous messaging using Apache Kafka
- Transaction management and ACID principles in a distributed microservices environment
- API testing and validation using Postman
- Version control and collaborative development using Git

Soft Skills

- Presentation skills — communicating technical work and outcomes clearly
- Problem-solving and analytical thinking
- Technical documentation and report writing
- Time management and meeting sprint deadlines
- Attention to detail during testing and validation
- Collaboration and teamwork within the project sprint cycle

Task Status Overview

Assignment	Task	Status	Completion Date	On Time	Overdue
EPIC 01	Task 01	Completed	26-08-2026	Yes	No
EPIC 01	Task 02	Completed	26-08-2026	Yes	No
EPIC 01	Task 03	Completed	26-08-2026	Yes	No
EPIC 01	Task 04	Completed	26-08-2026	Yes	No
EPIC 01	Task 05	Completed	27-08-2026	Yes	No
EPIC 02	Task 01	Completed	02-09-2026	Yes	No
EPIC 02	Task 02	Completed	02-09-2026	Yes	No
EPIC 02	Task 03	Completed	03-09-2026	Yes	No
EPIC 02	Task 04	Completed	03-09-2026	Yes	No
EPIC 02	Task 05	Completed	03-09-2026	Yes	No
EPIC 03	Task 01	Completed	04-09-2026	Yes	No
EPIC 03	Task 02	Completed	04-09-2026	Yes	No
EPIC 03	Task 03	Completed	04-09-2026	Yes	No
EPIC 03	Task 04	Completed	07-09-2026	Yes	No
EPIC 03	Task 05	Completed	09-09-2026	Yes	No
EPIC 04	Task 01	Completed	15-09-2026	Yes	No
EPIC 04	Task 02	Completed	16-09-2026	Yes	No
EPIC 04	Task 03	Completed	17-09-2026	Yes	No
EPIC 04	Task 04	Completed	18-09-2026	Yes	No
EPIC 04	Task 05	Completed	18-09-2026	Yes	No
EPIC 05	Task 01	Completed	21-09-2026	Yes	No
EPIC 05	Task 02	Completed	22-09-2026	Yes	No
EPIC 05	Task 03	Completed	23-09-2026	Yes	No
EPIC 05	Task 04	Completed	24-09-2026	Yes	No
EPIC 05	Task 05	Completed	24-09-2026	Yes	No
EPIC 06	Task 01	Completed	28-09-2026	Yes	No
EPIC 06	Task 02	Completed	29-09-2026	Yes	No
EPIC 06	Task 03	Completed	29-09-2026	Yes	No

EPIC 01 — Microservices Architecture Fundamentals

Assignment Overview

Assignment Start Date	26-08-2026
Assignment End Date / Deadline	28-08-2026
Overall Completion Date	27-08-2026
Overall Status	Completed
Completed Within Timeline	Yes
Overdue Status	No

Assignment Objective

Build a clear foundation in microservices architecture and apply the main concepts through a basic working system.

Detailed Task Record — Summary

Task	Task Name	Assigned Date	Due Date	Completion Date	Status
01	Monolithic vs Microservices Architecture	26-08-2026	28-08-2026	26-08-2026	Completed
02	Microservice Decomposition & Bounded Contexts	26-08-2026	28-08-2026	26-08-2026	Completed
03	Database-per-Service & Data Ownership	27-08-2026	28-08-2026	26-08-2026	Completed
04	Inter-Service Communication	27-08-2026	28-08-2026	26-08-2026	Completed
05	Basic Java Microservices System	28-08-2026	28-08-2026	27-08-2026	Completed

Task 01: Monolithic vs Microservices Architecture

Date: 26-08-2026

1. Objective

Understand the difference between monolithic and microservices architecture and identify why services are separated.

2. Work Completed

- Reviewed the main characteristics of both architecture styles.
- Created simple architecture diagrams.
- Worked with separate User, Product, and Order services.
- Compared deployment, scaling, coupling, and failure impact.

3. Validation

- Verified that the User, Product, and Order services could run independently.
- Checked the main service endpoints.

4. Outcome

Established a practical understanding of why independent services are useful and how they differ from a single monolithic application.

Task 02: Microservice Decomposition & Bounded Contexts

Date: 26-08-2026

1. Objective

Understand how a business system can be divided into focused and independent services.

2. Work Completed

- Used an e-commerce system as the example domain.
- Identified User, Product, Order, and Notification responsibilities.
- Defined service boundaries and responsibilities.
- Prepared a bounded-context and service-responsibility view.

3. Validation

- Checked the main service endpoints.
- Verified representative service operations.
- Resolved the Product Service port issue and confirmed successful startup.

4. Outcome

Created a clear service structure based on focused business responsibilities.

Task 03: Database-per-Service & Data Ownership

Date: 27-08-2026

1. Objective

Understand service-owned data and apply the database-per-service approach.

2. Work Completed

- Defined separate data ownership for the services.
- Documented that one service should not directly access another service's database.
- Implemented Payment Service with its own persistence structure.
- Configured separate database ownership and documented data access rules.

3. Validation

- Verified Payment Service startup.
- Tested payment creation and retrieval.
- Confirmed successful application and database initialization.

4. Outcome

Extended service separation to the data layer through clear database ownership and API-based access.

Task 04: Inter-Service Communication

Date: 27-08-2026

1. Objective

Understand how independent services communicate through REST APIs.

2. Work Completed

- Worked with separate User and Order services.
- Implemented the required APIs for the communication flow.
- Implemented service-to-service REST communication.
- Added error handling and documented the communication flow.

3. Validation

- Tested the services independently.
- Tested the REST communication between services.
- Checked user and order-related scenarios.

4. Outcome

Demonstrated practical communication between independent services without direct access to another service's internal layers or database.

Task 05: Basic Java Microservices System

Date: 28-08-2026

1. Objective

Combine the main microservices concepts into a simple working system.

2. Work Completed

- Created separate Spring Boot services.
- Implemented the required application layers and APIs.
- Configured service-owned persistence.
- Added inter-service communication, validation, and error handling.
- Prepared the architecture view and completed repository updates.

3. Validation

- Tested the main service endpoints.
- Tested the order flow with user lookup.
- Verified successful application execution.

4. Outcome

Completed a focused microservices system demonstrating independent services, service-owned data, communication, validation, and testing.

EPIC 02 — Spring Boot Microservices Foundation

Assignment Overview

Assignment Start Date	31-08-2026
Assignment End Date / Deadline	04-09-2026
Overall Completion Date	03-09-2026
Overall Status	Completed
Completed Within Timeline	Yes
Overdue Status	No

Assignment Objective

Improve a basic Spring Boot application into a clean and structured microservice foundation.

Detailed Task Record — Summary

Task	Task Name	Assigned Date	Due Date	Completion Date	Status
01	Production-Structured Spring Boot Microservice	31-08-2026	04-09-2026	02-09-2026	Completed
02	DTOs, Validation & Global Exception Handling	01-09-2026	04-09-2026	02-09-2026	Completed
03	Spring Boot Configuration & Environment Profiles	02-09-2026	04-09-2026	03-09-2026	Completed
04	Second Microservice & Service Boundaries	03-09-2026	04-09-2026	03-09-2026	Completed
05	Basic Multi-Microservice Architecture	04-09-2026	04-09-2026	03-09-2026	Completed

Task 01: Production-Structured Spring Boot Microservice

Date: 02-09-2026

1. Objective

Create a clean and maintainable application structure with a clear API flow.

2. Work Completed

- Created the required Spring Boot project structure.
- Separated controller, service, repository, entity, DTO, exception, and configuration responsibilities.
- Implemented the User domain and database operations.
- Implemented the required User API operations.

3. Validation

- Verified API behavior through Postman.
- Confirmed clear separation between API, business logic, and database access.

4. Outcome

Established a cleaner and more maintainable application structure.

Task 02: DTOs, Validation & Global Exception Handling

Date: 02-09-2026

1. Objective

Improve API input and output handling and provide consistent error responses.

2. Work Completed

- Created request and response DTOs.
- Removed direct exposure of internal entities.
- Added request validation rules.
- Created custom exceptions.
- Implemented centralized exception handling and standard error responses.

3. Validation

- Tested validation failures.
- Tested missing-resource and duplicate-data scenarios.
- Verified appropriate error responses and HTTP status codes.

4. Outcome

Improved API quality through cleaner data handling, validation, and consistent error management.

Task 03: Spring Boot Configuration & Environment Profiles

Date: 03-09-2026

1. Objective

Separate application configuration for different environments.

2. Work Completed

- Moved the main configuration approach to YAML.
- Created Development, Test, and Production profiles.
- Separated common and environment-specific settings.
- Reduced environment-specific configuration changes.

3. Validation

- Reviewed and verified the profile-based configuration structure.

4. Outcome

Created a clearer and more manageable configuration setup for different environments.

Task 04: Second Microservice & Service Boundaries

Date: 03-09-2026

1. Objective

Apply clear service boundaries and focused responsibilities across more than one service.

2. Work Completed

- Worked with a second microservice.
- Defined focused responsibilities for each service.
- Applied service ownership and database-per-service concepts.
- Followed interface-based communication instead of direct internal access.

3. Validation

- Verified separate service responsibilities and independent execution.

4. Outcome

Strengthened the transition from a single-service structure to a basic microservices architecture.

Task 05: Basic Multi-Microservice Architecture

Date: 03-09-2026

1. Objective

Build and test an architecture containing multiple services with separate responsibilities.

2. Work Completed

- Included multiple Spring Boot services.
- Maintained clear ownership boundaries.
- Tested the main APIs.
- Used health checks to support application availability verification.

3. Validation

- Tested successful and invalid API requests.
- Tested missing-resource, duplicate, and validation scenarios.
- Verified HTTP status codes and health checks.

4. Outcome

Completed a tested multi-service foundation with independent responsibilities and improved API quality.

EPIC 03 — REST API Design & Professional API Standards

Assignment Overview

Assignment Start Date	04-09-2026
Assignment End Date / Deadline	11-09-2026
Overall Completion Date	09-09-2026
Overall Status	Completed
Completed Within Timeline	Yes
Overdue Status	No

Assignment Objective

Build a professional Product Service API with clear REST design, validation, data handling, and documentation.

Detailed Task Record — Summary

Task	Task Name	Assigned Date	Due Date	Completion Date	Status
01	REST API Fundamentals & HTTP Standards	04-09-2026	11-09-2026	04-09-2026	Completed
02	DTOs, Request Validation & API Error Standards	04-09-2026	11-09-2026	04-09-2026	Completed
03	Pagination, Sorting & Filtering	04-09-2026	11-09-2026	04-09-2026	Completed
04	API Versioning & Backward Compatibility	04-09-2026	11-09-2026	07-09-2026	Completed
05	Swagger/OpenAPI & Professional API Documentation	04-09-2026	11-09-2026	09-09-2026	Completed

Task 01: REST API Fundamentals & HTTP Standards

Date: 04-09-2026

1. Objective

Understand REST principles and implement properly structured APIs.

2. Work Completed

- Created the Product Service.
- Implemented the required product data model.
- Implemented CRUD APIs using the correct HTTP methods.
- Applied suitable HTTP status codes.

3. Validation

- Tested successful requests and invalid scenarios in Postman.
- Checked create, read, update, and delete behavior.

4. Outcome

Established a standards-based REST API foundation.

Task 02: DTOs, Request Validation & API Error Standards

Date: 04-09-2026

1. Objective

Create clean request models, validate input, and return consistent error responses.

2. Work Completed

- Added request and response DTOs.
- Added validation for product input fields.
- Implemented centralized exception handling.
- Standardized validation and error responses.

3. Validation

- Tested invalid product input.
- Verified validation responses.
- Tested not-found, duplicate, and generic error scenarios.

4. Outcome

Created a cleaner and more predictable API contract.

Task 03: Pagination, Sorting & Filtering

Date: 04-09-2026

1. Objective

Support efficient handling of larger sets of product data.

2. Work Completed

- Implemented pagination.
- Added sorting options.
- Added category and price-range filtering.
- Combined filtering, sorting, and pagination.
- Added query parameter validation.

3. Validation

- Tested pagination results and metadata.
- Tested sorting and filtering options.
- Verified invalid parameters return clean error responses.

4. Outcome

Improved the API's ability to handle and search larger product datasets.

Task 04: API Versioning & Backward Compatibility

Date: 07-09-2026

1. Objective

Understand how an API can evolve without breaking existing clients.

2. Work Completed

- Reviewed common API versioning approaches.
- Kept the existing v1 API available.
- Introduced v2 with additional response information.
- Documented the difference between the versions and the migration approach.

3. Validation

- Verified v1 responses.
- Verified v2 responses.
- Confirmed that both versions work independently.

4. Outcome

Demonstrated a backward-compatible approach to API evolution.

Task 05: Swagger/OpenAPI & Professional API Documentation

Date: 09-09-2026

1. Objective

Create clear API documentation that supports understanding and testing without reading the source code.

2. Work Completed

- Added Swagger/OpenAPI support.
- Documented endpoints, parameters, request bodies, and responses.
- Added realistic request and response examples.
- Grouped APIs clearly and reviewed the overall API design.

3. Validation

- Executed and verified endpoints through Swagger UI.
- Confirmed documented behavior matched the implemented API.

4. Outcome

Completed clear and usable API documentation for understanding and testing the Product Service.

EPIC 04 — API Security & Authentication with Spring Security

Assignment Overview

Assignment Start Date	15-09-2026
Assignment End Date / Deadline	18-09-2026
Overall Completion Date	18-09-2026
Overall Status	Completed
Completed Within Timeline	Yes
Overdue Status	No

Assignment Objective

Learn and implement API security for Java Spring Boot microservices, including authentication, authorization, password hashing, JWT access and refresh tokens, role-based access control, secured endpoints, and security testing. The sprint covered 4 working days with 40 total hours and 5 Jira tasks at 8 hours each; Task 05 served as the final integration task.

Detailed Task Record — Summary

Task	Task Name	Assigned Date	Due Date	Completion Date	Status
01	Spring Security Fundamentals & Authentication Setup	15-09-2026	15-09-2026	15-09-2026	Completed
02	User Registration, Password Hashing & Login API	16-09-2026	16-09-2026	16-09-2026	Completed
03	JWT Access Token & Refresh Token Implementation	17-09-2026	17-09-2026	17-09-2026	Completed
04	Role-Based Authorization & API Permissions	18-09-2026	18-09-2026	18-09-2026	Completed
05	Complete Secure Microservice Integration & Security Audit	18-09-2026	18-09-2026	18-09-2026	Completed

Task 01: Spring Security Fundamentals & Authentication Setup

Date: 15-09-2026

1. Objective

Understand Spring Security fundamentals and configure a basic security layer for a Spring Boot REST API.

2. Work Completed

- Studied authentication, authorization, identity, roles, permissions, access control, and public versus protected APIs.
- Reviewed the security request flow: client → security filter → authentication → authorization → controller.
- Added Spring Security to the existing Product/User microservice.
- Created a SecurityFilterChain and defined public and protected API rules.
- Created a public endpoint at GET /api/v1/products/public and protected private product operations.

3. Validation

- Ran the application after adding security configuration.
- Called protected APIs without credentials and verified the expected 401 Unauthorized behavior.
- Reviewed the security flow and documented the public/private API configuration.

4. Outcome

Established the basic Spring Security layer and API boundary for subsequent authentication and authorization work.

Task 02: User Registration, Password Hashing & Login API

Date: 16-09-2026

1. Objective

Build a secure user registration and login mechanism.

2. Work Completed

- Created the User model with id, name, email, password, role, status, and createdAt fields.
- Created RegisterRequest containing name, email, and password.
- Added registration validation for required name, valid email, required password, and minimum password requirements.
- Configured BCryptPasswordEncoder and ensured passwords are not stored in plain text.
- Implemented POST /api/v1/auth/register and duplicate-email handling with 409 Conflict.
- Implemented POST /api/v1/auth/login and credential verification through the user service.

3. Validation

- Tested successful registration and login requests.
- Tested invalid registration input and duplicate-user behavior.
- Verified incorrect credentials produce 401 Unauthorized.
- Checked the database to confirm the stored password is hashed rather than the original password.

4. Outcome

Completed the secure registration and login foundation required for JWT-based authentication.

Task 03: JWT Access Token & Refresh Token Implementation

Date: 17-09-2026

1. Objective

Implement JWT-based authentication for the microservice API.

2. Work Completed

- Studied JWT header, payload, signature, access token, refresh token, expiration, and stateless authentication.
- Configured a secret key and separate access-token and refresh-token expiration settings without hard-coding secrets in source code.
- Generated JWTs after successful login.
- Added claims such as userId, email, role, issuedAt, and expiration.
- Created an authentication response containing accessToken, refreshToken, tokenType, and expiresIn.
- Created a JWT authentication filter that reads the Authorization header, extracts the Bearer token, validates it, extracts user information, and sets the SecurityContext.
- Implemented POST /api/v1/auth/refresh to generate a new access token after validating the refresh token.

3. Validation

- Tested successful login and token generation.
- Tested protected API access with a valid Bearer token.
- Tested missing, invalid, expired, and modified token scenarios.
- Verified the refresh-token endpoint returns a new access token.

4. Outcome

Implemented stateless JWT access and refresh token authentication for protected APIs.

Task 04: Role-Based Authorization & API Permissions

Date: 18-09-2026

1. Objective

Configure role-based authorization and API permissions for USER, MANAGER, and ADMIN roles.

2. Work Completed

- Defined USER, MANAGER, and ADMIN roles.
- Assigned USER as the default role during registration and prevented a normal user from selecting ADMIN.
- Configured Spring Security authorization rules based on roles and API operations.
- Defined permissions for viewing, creating, updating, deleting products, and managing users.
- Documented the difference between authentication and authorization using 401 versus 403 responses.

3. Validation

- Verified USER can view products but cannot create, update, or delete products or manage users.
- Verified MANAGER can view, create, and update products and has the configured manager permissions.
- Verified ADMIN can perform the full set of configured product and user-management operations.
- Tested an unauthorized role attempting an ADMIN API and verified 403 Forbidden.

4. Outcome

Applied explicit role-based access rules and verified permission boundaries across the API.

Task 05: Complete Secure Microservice Integration & Security Audit

Date: 18-09-2026

1. Objective

Integrate the security capabilities from the Epic into a complete secured Spring Boot microservice and perform a final security audit.

2. Work Completed

- Integrated registration, login, BCrypt hashing, JWT access/refresh tokens, JWT filtering, roles, permissions, and secured APIs.
- Applied the final architecture flow: client → authentication → JWT token → Spring Security → user/admin API → database.
- Reviewed the complete registration and authentication flow.
- Reviewed API security rules, error handling, roles, permissions, and security checklist items.
- Prepared the final demonstration flow covering registration, login, JWT receipt, protected API access, role validation, rejection handling, and token refresh.

3. Validation

- Verified the integrated secure API workflow end to end.
- Verified protected access with valid authentication and rejected requests with invalid or missing security context.
- Reviewed 401 and 403 handling as part of the security audit.
- Prepared the supporting Postman collection, security documentation, and final demonstration evidence.

4. Outcome

Completed the secured microservice integration and security audit for the Epic.

Additional Note

Deliverables covered the secured Spring Boot API, registration, login, BCrypt hashing, JWT access/refresh tokens, JWT filter, role-based authorization, 401/403 handling, security audit, SECURITY.md, Postman collection, repository updates, and final demo.

EPIC 05 — Inter-Service Communication & Resilience

Assignment Overview

Assignment Start Date	21-09-2026
Assignment End Date / Deadline	25-09-2026
Overall Completion Date	24-09-2026
Overall Status	Completed
Completed Within Timeline	Yes
Overdue Status	No

Assignment Objective

Learn how independent microservices communicate with each other and how to make those communications reliable when services are slow, unavailable, or failing. All five tasks were completed within the sprint timeline.

Detailed Task Record — Summary

Task	Task Name	Assigned Date	Due Date	Completion Date	Status
01	REST-Based Inter-Service Communication	21-09-2026	21-09-2026	21-09-2026	Completed
02	Timeout, Retry & Failure Handling	22-09-2026	22-09-2026	22-09-2026	Completed
03	Circuit Breaker & Fallback	23-09-2026	23-09-2026	23-09-2026	Completed
04	Asynchronous Communication with Apache Kafka	24-09-2026	24-09-2026	24-09-2026	Completed
05	Complete Resilient Microservices Workflow	25-09-2026	25-09-2026	24-09-2026	Completed

Task 01: REST-Based Inter-Service Communication

Date: 21-09-2026

1. Objective

Implement communication between two independent Spring Boot microservices using REST.

2. Work Completed

- Studied inter-service communication, synchronous versus asynchronous communication, REST-based communication, request-response flow, and service dependency.
- Worked with User Service on port 8081 and Order Service on port 8082.
- Implemented the User API GET `/api/v1/users/{id}`.
- Implemented the Order API GET `/api/v1/orders/{id}` and the required order fields.
- Implemented the Order Service calling the User Service through a Spring HTTP client rather than direct database access.
- Created a combined order response containing order information and user information.
- Prepared a sequence flow showing client → Order Service → User Service → response → Order Service → client.

3. Validation

- Verified that user and order records exist and their individual APIs respond.
- Verified Order Service receives user information through REST service-to-service communication.
- Tested the communication flow and documented the sequence and failure observation path.

4. Outcome

Demonstrated independent microservices communicating through a REST API without direct access to another service database.

Task 02: Timeout, Retry & Failure Handling

Date: 22-09-2026

1. Objective

Handle situations where a dependent microservice is slow or temporarily unavailable.

2. Work Completed

- Studied connection timeout, read timeout, request timeout, and why indefinite waiting is dangerous.
- Configured a reasonable timeout for the Order Service → User Service call.
- Tested timeout behavior by making the User Service intentionally slow.
- Studied temporary failures, retry necessity, maximum retry attempts, retry delay, and exponential backoff.
- Implemented limited retries for appropriate transient failures.
- Configured retry limits and distinguished retryable from non-retryable errors.
- Added logging for request started, retry attempt, retry failure, and final failure without logging secrets.

3. Validation

- Verified the Order Service does not wait indefinitely when the User Service is slow.
- Simulated temporary User Service failure and observed retries according to policy.
- Verified final failure results in a controlled error response after the configured retry limit.

4. Outcome

Added bounded timeout and retry behavior to make synchronous service communication more resilient to slow and transient failures.

Task 03: Circuit Breaker & Fallback

Date: 23-09-2026

1. Objective

Implement the Circuit Breaker pattern to prevent cascading failures between microservices.

2. Work Completed

- Studied CLOSED, OPEN, and HALF_OPEN circuit breaker states and the state-transition flow.
- Added the required resilience library and applied Circuit Breaker protection to the Order Service → User Service call.
- Configured failure threshold, sliding window, wait duration, and permitted test calls using development-appropriate values.
- Created a fallback response for User Service unavailability, including orderId, product, user=null, and an unavailable-service message.
- Tested healthy service behavior, stopped the User Service to trigger failure handling, and restarted it to observe recovery.

3. Validation

- Verified normal Order → User communication while the dependency is healthy.
- Generated repeated failures and observed the circuit breaker opening.
- Restarted the User Service and verified the transition toward HALF_OPEN and then CLOSED after successful calls.
- Verified the fallback response is returned when the dependency is unavailable.

4. Outcome

Added circuit breaking and fallback behavior to reduce repeated downstream calls and isolate dependency failures.

Task 04: Asynchronous Communication with Apache Kafka

Date: 24-09-2026

1. Objective

Understand event-driven architecture and implement asynchronous communication using Apache Kafka.

2. Work Completed

- Studied Kafka messaging concepts: message, event, producer, consumer, topic, partition, consumer group, and offset.
- Compared synchronous request-response communication with asynchronous event delivery.
- Configured Kafka-based asynchronous communication and the required topic flow.
- Implemented producer and consumer components and the order event model.
- Implemented the order event flow from Order Service to Kafka and onward to Notification Service.
- Documented the event-driven architecture and processing flow.

3. Validation

- Created an order and verified Kafka receives the event.
- Verified Notification Service receives and processes the event.
- Stopped Notification Service, created an order, restarted Notification Service, and verified pending-event handling according to the configured consumer behavior.

4. Outcome

Demonstrated asynchronous event-driven communication with Kafka and a consumer that processes order events independently of the producing service.

Task 05: Complete Resilient Microservices Workflow

Date: 25-09-2026

1. Objective

Combine REST communication, timeout, retry, Circuit Breaker, fallback, and Kafka into one complete microservices workflow.

2. Work Completed

- Integrated the communication and resilience patterns developed across the Epic into a single workflow.
- Combined synchronous REST service calls with timeout, retry, Circuit Breaker, and fallback handling.
- Included asynchronous Kafka event communication in the end-to-end architecture.
- Validated the overall resilient microservice workflow and prepared the final architecture evidence and documentation.

3. Validation

- Verified the combined architecture flow across synchronous and asynchronous paths.
- Reviewed resilient behavior for slow, transient, and unavailable dependencies.
- Verified the final workflow and supporting documentation as the Epic completion activity.

4. Outcome

Completed the final resilient microservices workflow and closed EPIC 05 within the sprint timeline.

Additional Note

Although Task 05 was scheduled for 25-09-2026, it was completed on 24-09-2026, one day early; therefore the task remained within the planned timeline with no overdue work.

EPIC 06 — Database & Distributed Data Design

Assignment Overview

Assignment Start Date	28-09-2026
Assignment End Date / Deadline	30-09-2026
Overall Completion Date	29-09-2026
Overall Status	Completed
Completed Within Timeline	Yes
Overdue Status	No

Assignment Objective

Design and optimize service-owned databases, improve query performance, and implement transaction fundamentals for reliable microservice data operations.

Detailed Task Record — Summary

Task	Task Name	Assigned Date	Due Date	Completion Date	Status
01	Database Design & Database-per-Service	28-09-2026	28-09-2026	28-09-2026	Completed
02	Database Indexing, Query Optimization & Performance	29-09-2026	29-09-2026	29-09-2026	Completed
03	Transactions & Distributed Transaction Fundamentals	30-09-2026	30-09-2026	29-09-2026	Completed

All three EPIC 06 tasks were completed within the planned timeline. Task 03 was completed on 29-09-2026, one day before its scheduled 30-09-2026 date.

Task 01: Database Design & Database-per-Service Architecture

Date: 28-09-2026

1. Objective

Implement and verify clear database ownership for the core microservices and align the persistence model with the database-per-service architecture.

2. Work Completed

- Defined dedicated databases for User, Product, Order, and Payment services.
- Verified user_service_dev_db, productdb, order_service_db, and paymentdb ownership.
- Created order_items in order_service_db with a foreign-key relationship to orders.
- Added the OrderItem JPA entity and repository to persist order-item records.
- Aligned Payment Service persistence with payment_status, transaction_reference, and created_at.
- Verified Order Service communicates with User Service through its API instead of direct database access.
- Verified the existing Kafka order-event flow to Notification Service.

3. Validation

- Verified service configurations and database connections.
- Created an order successfully through POST /api/v1/orders and verified orders and order_items in MySQL.
- Verified Payment Service creation and the resulting payment row in paymentdb.
- Verified Kafka order-created processing by Notification Service.

4. Outcome

Established clear service-owned data boundaries and database-per-service persistence across the core services. The task was completed on 28-09-2026 with no overdue status.

Task 02: Database Indexing, Query Optimization & Performance

Date: 29-09-2026

1. Objective

Improve Order Service database query efficiency through indexing, query optimization, pagination, and execution-plan analysis.

2. Work Completed

- Expanded the orders table from 13 records to exactly 10,000 records for performance testing.
- Captured a baseline EXPLAIN plan for filtering by user_id before indexing.
- Created the user_id_index single-column index.
- Created the user_created_index composite index on user_id and created_at.
- Optimized the query to select only required columns instead of SELECT *.
- Implemented and verified LIMIT/OFFSET pagination.
- Used EXPLAIN ANALYZE to inspect actual execution behavior.
- Kept indexing focused on demonstrated query patterns rather than adding unnecessary indexes.

3. Validation

- Before indexing, EXPLAIN showed type ALL, key NULL, with approximately 9,643 estimated rows.
- After user_id_index, EXPLAIN used type ref and estimated about 1,010 rows.
- The composite query used user_created_index with a Backward index scan.
- Page 1 and Page 2 pagination were verified with 10 records per page.
- EXPLAIN ANALYZE confirmed the optimized index-backed execution path.

4. Outcome

Established a measurable database optimization workflow using a 10,000-row dataset: baseline plan, index design, optimized query, pagination, and execution-plan verification. The task was completed on 29-09-2026 with no overdue status.

Task 03: Transactions & Distributed Transaction Fundamentals

Date: 30-09-2026

1. Objective

Implement local transaction management in Order Service, demonstrate rollback, understand ACID properties, and analyze transaction boundaries across microservices.

2. Work Completed

- Added Spring @Transactional to OrderService.createOrder().
- Kept the existing Order and OrderItem JPA/repository workflow and made both writes part of the local transaction.
- Verified the successful commit path by creating an order and confirming its OrderItem in MySQL.
- Added a temporary controlled RuntimeException after the database writes using productName = FAIL_TRANSACTION.
- Verified the failure returned HTTP 500 and that both the orders and order_items queries returned 0 rows, demonstrating rollback.
- Removed the temporary failure code and restarted Order Service with the clean implementation.
- Documented Atomicity, Consistency, Isolation, and Durability.
- Analyzed distributed boundaries involving User Service REST communication and Kafka/Notification processing.

3. Validation

- Normal POST /api/v1/orders returned HTTP 201 Created and Order ID 16397 was verified with its corresponding order_items record.
- FAIL_TRANSACTION returned HTTP 500 Internal Server Error.
- MySQL confirmed no FAIL_TRANSACTION rows remained in either orders or order_items.
- Service was restored after removing the temporary rollback test code.

4. Outcome

Completed the local transaction and rollback implementation and documented the difference between local ACID transactions and distributed transaction boundaries.

Additional Note

Task 03 was completed on 29-09-2026, one day before the scheduled 30-09-2026 date, with no overdue status.